

Algoritmos y Estructuras de Datos II

Análisis de Complejidad por casos

Cristian Fernandez

1. Demuestre que $6n^3 \neq O(n^2)$.

La definición de Big-Oh nos dice: Sea una función $f(n)$, decimos que $f(n)$ pertenece a la familia de funciones $O(g(n))$ si existe una constante $C1$ y un número n_0 tales que:

$$0 \leq f(n) \leq C1 * g(n) \text{ para } n \geq n_0$$

Reemplazando nos queda

$$0 \leq 6n^3 \leq C1 * n^2$$

Lo cual no es posible porque esa constante $C1$ tendría que ser mayor o igual que $6n$, como su valor depende del valor de n entonces no es una constante y no cumple con la definición.

2. ¿Cómo sería un array de números (mínimo 10 elementos) para el mejor caso de la estrategia de ordenación Quicksort(n)?

Una posible distribución de elementos que provoque el mejor caso en Quicksort(n) podría ser:

5	2	4	7	9	1	3	0	8	6
---	---	---	---	---	---	---	---	---	---

Con esta distribución cada vez que se elige al primer elemento como pivote, la lista se dividirá en dos subproblemas cuyo tamaño siempre será menor o igual a $n/2$, una de tamaño $\lfloor n/2 \rfloor$ y otra de tamaño $\lfloor n/2 \rfloor - 1$:

Una sublista de 5 elementos y otra de 4:

2	4	1	3	0	5	7	9	8	6
---	---	---	---	---	---	---	---	---	---

Dos sublistas de 2 elementos (con las cuales se repetirá el proceso, pero al tener solo dos elementos será mucho más sencillo):

1	0	2	4	3
---	---	---	---	---

Una sublista de 1 elemento y otra de 2 (de nuevo, se repetirá el proceso):

6	7	9	8
---	---	---	---

3. ¿Cuál es el tiempo de ejecución de la estrategia Quicksort(A), Insertion-Sort(A) y Merge-Sort(A) cuando todos los elementos del array A tienen el mismo valor?

Quicksort(A): como todos los elementos son iguales la división de subproblemas siempre será desbalanceada porque habrá una de las sublistas que estará vacía, por lo tanto, el tiempo de ejecución sería $O(n^2)$

Insertion-Sort(A): al hacer la comparación entre los elementos nunca va a realizar el intercambio debido a que los elementos serán iguales, por lo tanto, lo único que hará será compararlos y su tiempo de ejecución sería $O(n)$

Merge-Sort(A): este algoritmo no depende de cuáles sean sus elementos ya que siempre tiene el mismo procedimiento (dividir y luego unir), así, su tiempo de ejecución será $O(n \log n)$

4. Implementar un algoritmo que ordene una lista de elementos donde siempre el elemento del medio de la lista contiene antes que él en la lista la mitad de los elementos menores que él. Explique la estrategia de ordenación utilizada.

Ejemplo de lista de salida:

7	3	2	8	5	4	1	6	10	9
---	---	---	---	---	---	---	---	----	---

```
def ordenar_elemento_medio(A: List) -> List:
    medio = A[(len(A) // 2) - 1] if len(A) % 2 == 0 else A[len(A) // 2]
    menores = []
    otros = []
    for elemento in A:
        if elemento < medio:
            menores.append(elemento)
        elif elemento != medio:
            otros.append(elemento)
    mitad = len(menores) // 2
    menoresAntes = menores[:mitad]
    menoresDespues = menores[mitad:]
    elementosAntes = len(A)//2 - 1 if len(A) % 2 == 0 else len(A)//2
    izq = menoresAntes + otros[:elementosAntes - len(menoresAntes)]
    der = menoresDespues + otros[elementosAntes - len(menoresAntes):]
    return izq + [medio] + der
```

El algoritmo busca primero el elemento que se encuentra al medio (siendo $n//2-1$ si es par y $n//2$ si es impar). Luego guarda en una lista aquellos elementos que sean menores al elemento central y los otros en otra lista. Luego calcula cuál es la mitad de elementos menores (que son los que tendrán que ir antes) y divide la lista en dos partes a la mitad. Después calcula cuántos elementos tendrán que ir antes del elemento central para que siga estando en el medio y coloca la primera mitad de menores antes completándola con los otros para tener la cantidad de elementos necesarias a la izquierda del medio. Por último, completa con el elemento central y a la derecha con el resto de números.

Este algoritmo tiene complejidad temporal y espacial $O(n)$ ya que realiza varios recorridos a la lista, pero todos de manera lineal, y nunca anidadas, y utiliza listas auxiliares para dividir los elementos.

5. Implementar un algoritmo **Contiene-Suma(A,n)** que recibe una lista de enteros A y un entero n y devuelve True si existen en A un par de elementos que sumados den n. Analice el costo computacional.

```
def ContieneSuma(L,n):
    if L.head is None or L.head.nextNode is None:
        return False
    current = L.head
    while current:
        target = n - current.value
        p = current.nextNode
        while p:
            if p.value == target:
                return True
            p = p.nextNode
        current = current.nextNode
    return False
```

Para cada elemento de la lista, el algoritmo busca entre los elementos siguientes alguno cuyo valor sea igual a n menos el valor del elemento actual. En el peor caso, cuando no existe ningún par que sume n , se deben comparar todos los pares de elementos siendo el total de operaciones $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} \rightarrow O(n^2)$. En el mejor caso los dos primeros elementos sumarían n , entonces no debería recorrer nunca toda la lista y su complejidad sería $\Omega(1)$. En el caso promedio, al igual que en el peor caso sería de $\Theta(n^2)$ debido a que tiene dos bucles *while* anidados.

- 6. Investigar otro algoritmo de ordenamiento como BucketSort, HeapSort o RadixSort, brindando un ejemplo que explique su funcionamiento en un caso promedio. Mencionar su orden y explicar sus casos promedio, mejor y peor.**

HeapSort es un algoritmo de ordenamiento que utiliza una estructura llamada heap, que puede representarse como un árbol binario. Para ordenar de menor a mayor, se construye un maxHeap, donde el elemento mayor se encuentra en la raíz. Luego, se intercambia ese elemento con el último de la lista y se vuelve a reorganizar el heap. Este proceso se repite hasta obtener la lista ordenada.

Por ejemplo, si tenemos la lista: [4, 10, 3, 5, 1]. Primero se construye el maxHeap, colocando el 10 en la raíz. Luego se intercambia el mayor con el último elemento y se reorganiza el heap.

Repitiendo este procedimiento se obtiene finalmente:

[10, 5, 3, 4, 1] -> [1, 5, 4, 3, 10]

[5, 4, 3, 1, 10] -> [1, 4, 3, 5, 10]

[4, 1, 3, 5, 10] -> [3, 1, 4, 5, 10]

[3, 1, 4, 5, 10] -> [1, 3, 4, 5, 10]

El orden de HeapSort es: $O(n \log n)$ en el mejor, promedio y peor caso. Esto se debe a que la construcción del heap puede realizarse en $O(n)$, mientras que se realizan aproximadamente (n) extracciones, y cada una puede requerir $(O(\log n))$.

Una ventaja de HeapSort es que mantiene este orden incluso en el peor caso, a diferencia de Quicksort, que puede llegar a $(O(n^2))$.

- 7. A partir de las siguientes ecuaciones de recurrencia, encontrar la complejidad expresada en $\Theta(n)$ y ordenarlas de forma ascendente respecto a la velocidad de crecimiento.**

Asumiendo que $T(n)$ es constante para $n \leq 2$. Resolver 3 de ellas con el método maestro completo: $T(n) = a T(n/b) + f(n)$ y otros 3 con el método maestro simplificado: $T(n) = a T(n/b) + n^c$

a) $T(n) = 2T(n/2) + n^4$

b) $T(n) = 2T(7n/10) + n$

c) $T(n) = 16T(n/4) + n^2$

d) $T(n) = 7T(n/3) + n^2$

e) $T(n) = 7T(n/2) + n^2$

f) $T(n) = 2T(n/4) + \sqrt{n}$

Método Maestro Completo:

a) $a = 2; b = 2; f(n) = n^4 \rightarrow n^{\log_2 2} = n^1 = \theta(n)$

Entra en el caso 3, ya que con $\varepsilon=3$, $f(n) = n^{\log_b a + \varepsilon}$, y además comprobamos que

$$af\left(\frac{n}{b}\right) \leq cf(n)$$

$$2\left(\frac{n}{2}\right)^4 \leq cn^4$$

$$\frac{1}{8} \leq c$$

$$\text{Así, } T(n) = \theta(n^4)$$

$$\text{b) } a = 2; b = \frac{10}{7}; f(n) = n \rightarrow n^{\log_{10} 2} \approx n^{1.94} = \theta(n^2)$$

Entra en el caso 1, ya que con $\varepsilon \approx 1$, $f(n) = n^{\log_b a - \varepsilon}$

$$\text{Así, } T(n) = \theta(n^2)$$

$$\text{c) } a = 16; b = 4; f(n) = n^2 \rightarrow n^{\log_4 16} = n^2 = \theta(n^2)$$

Entra en el caso 2, ya que $f(n) = n^{\log_b a}$

$$\text{Así, } T(n) = \theta(n^2 \log n)$$

Método Maestro Simplificado:

$$\text{d) } a = 7; b = 3; c = 2; p = \log_3 7 \approx 1.77$$

Como $p < c$, entra en el caso 3, y además comprobamos que

$$7\left(\frac{n}{3}\right)^2 \leq kn^2$$

$$\frac{7}{9} \leq k$$

$$\text{Así, } T(n) = \theta(n^2)$$

$$\text{e) } a = 7; b = 2; c = 2; p = \log_2 7 \approx 2.81$$

Como $p > c$, entra en el caso 1

$$\text{Así, } T(n) = \theta(n^{\log_2 7})$$

$$\text{f) } a = 2; b = 4; c = \frac{1}{2}; p = \log_4 2 = \frac{1}{2}$$

Como $p = c$, entra en el caso 2

$$\text{Así, } T(n) = \theta(\sqrt{n} \log n)$$

Ordenándolas de forma ascendente por velocidad de crecimiento quedaría así:

$$\theta(\sqrt{n} \log n) \rightarrow \theta(n^2 \log n) \rightarrow \theta(n^2) \rightarrow \theta(n^{\log_2 7}) \rightarrow \theta(n^4)$$